TP – RESSOURCES AVANCÉES & INJECTION (LARAVEL)

Comprendre simplement COMMENT Laravel relie :
URL → route → contrôleur → paramètres → objets injectés

PARTIE 1 — RESSOURCES IMBRIQUÉES

Un post appartient à un user
Donc l'URL DOIT contenir l'ID du user

🗀 **routes/web.php**

```
Route::resource('users.posts', PostController::class);
```

EXPLICATION

- users.posts :
  - users = ressource parent
  - posts = ressource enfant
- Laravel comprend :

« Un post est lié à un user »

Laravel génère automatiquement des routes comme :

/users/{user}/posts

/users/{user}/posts/{post}

On crée le controleur **PostController**

🗀 **PostController.php**

```
class PostController extends Controller
{
    // Cette méthode correspond à :
```

```
    // GET /users/{user}/posts
    public function index($user)
    {
        return "LISTE DES POSTS DU USER ID = $user";
    }

    // Cette méthode correspond à :
    // GET /users/{user}/posts/{post}
    public function show($user, $post)
    {
        return "POST ID = $post DU USER ID = $user";
    }
}
```

EXPLICATION

- $user vient de {user} dans l'URL
- $post vient de {post} dans l'URL
- Laravel lit l'URL et **injecte les valeurs automatiquement**

"Pourquoi deux paramètres ?"

Réponse simple :

Parce que l'URL a deux variables

TESTER

Dans le navigateur :

http://localhost:8000/users/1/posts

http://localhost:8000/users/1/posts/3

PARTIE 2 — NIDIFICATION PEU PROFONDE (SHALLOW)

Shallow (nidification peu profonde)

Quand on connaît l'ID du post,
on n'a PLUS besoin de l'ID du user

🗀 **routes/web.php**

```
Route::resource('users.posts', PostController::class)->shallow();
```

CE QUE FAIT LARAVEL

- Pour **index** :

/users/{user}/posts

- Pour **show** :

/posts/{post}

URL plus courte
Plus lisible
Plus logique

🗀 **PostController.php**

```
class PostController extends Controller
{
    // GET /users/{user}/posts
    public function index($user)
```

```
    {
        return "POSTS DU USER ID = $user";
    }

    // GET /posts/{post}
    public function show($post)
    {
        return "POST ID = $post (SHALLOW)";
    }
}
```

TEST

/users/2/posts

/posts/7

-----Shallow = simplifier l'URL quand c'est possible

PARTIE 3 — RENOMMAGE DES ROUTES

Renommage des routes

Le nom de la route est pour le développeur
L'URL est pour l'utilisateur

🗀 **routes/web.php**

```
Route::resource('users', UserController::class)->names([
    'create' => 'users.build',
    'edit' => 'users.modify',
```

```
]);
```

CE QUE LARAVEL FAIT

Méthode	Nom route
create	users.build
edit	users.modify

🗀 **UserController.php**

```
class UserController extends Controller
{
    public function create()
    {
        return "FORMULAIRE CREATION USER";
    }

    public function edit($id)
    {
        return "FORMULAIRE MODIFICATION USER ID = $id";
    }
}
```

IMPORTANT

- L'URL reste :

/users/create

/users/3/edit

------Seul le nom interne change

□ PARTIE 4 — PERSONNALISATION DES PARAMÈTRES

-- Changer le nom du paramètre

On peut renommer les variables pour mieux comprendre le code

🗀 routes/web.php

```
Route::resource('users', UserController::class)->parameters([
    'users' => 'admin_user'
]);
```

CE QUE LARAVEL COMPREND

Avant :

show($id)

Après :

show($admin_user)

🗀 UserController.php

```
class UserController extends Controller
{
    public function show($admin_user)
    {
        return "ADMIN USER ID = $admin_user";
```

```
    }
}
```

TEST

/users/5

L'URL ne change PAS
Seul le nom de la variable change

PARTIE 5 — INJECTION DE DÉPENDANCES

----Laravel crée les objets à ta place

Injection par constructeur

🗀 **Créer un Repository**

php artisan **make:class Repositories/UserRepository**

Qu'est-ce qu'un Repository ?

Définition simple :

Un **repository** est une **classe qui centralise l'accès aux données**.
Elle sait comment récupérer, créer, modifier ou supprimer des données, mais **le contrôleur n'a pas besoin de savoir comment ça fonctionne à l'intérieur**.

Exemple :

- Imagine une entreprise :
  - Le **contrôleur** = le chef qui donne les instructions
  - Le **repository** = le bureau où sont stockées toutes les informations

- Le chef ne fouille pas dans les dossiers, il demande au bureau :

« Donne-moi la liste des utilisateurs »

- Le bureau (repository) renvoie la liste
- Le chef (contrôleur) peut maintenant afficher ou modifier ces utilisateurs

Pourquoi utiliser un repository ?

- Séparer la **logique métier** de la **logique de récupération des données**
- Facilite les **tests** (on peut remplacer le repository par un faux)
- Facilite la **maintenance** (si demain on change la base de données, le contrôleur ne change pas)
- C'est une bonne pratique dans **les architectures propres (Clean Architecture)**

🗀 **UserRepository.php**

```
namespace App\Repositories;


class UserRepository
{
// Méthode pour récupérer tous les utilisateurs
   public function getAll()
   {
      // Simule une base de données
      return ['Ali', 'Sara', 'Hamid'];
   }
 }
```

---Comment le contrôleur l'utilise ?

🗀 **UserController.php**

```
use App\Repositories\UserRepository;

class UserController extends Controller
{
   protected $users;
  // Laravel injecte UserRepository automatiquement
   public function __construct(UserRepository $users)
   {
      $this->users = $users;
   }

   public function index()
   {
      return $this->users->getAll();// Affiche juste le tableau pour test
   }
}
```

EXPLICATION

- Laravel voit UserRepository
- Il crée l'objet automatiquement
- Il l'envoie au constructeur

- **Aucun new écrit par le développeur**

Points à connaitre :

1. **Le repository fait le travail "d'accès aux données"**
2. **Le contrôleur ne se soucie pas de la manière dont les données sont stockées**
3. **Si demain on met une vraie base de données, le contrôleur ne change pas**
4. **Laravel crée automatiquement le repository grâce à l'injection de dépendances**

---- Injection par méthode

Concept simple

Laravel **peut fournir automatiquement des objets aux méthodes d'un contrôleur**, simplement en regardant **le type de la variable** dans la signature de la méthode.

- Ici, on utilise Request $request
- Laravel crée l'objet Request automatiquement et **le passe à la méthode**

Exemple :

- Imagine une lettre que tu envoies à ton chef :
  - Le chef n'a pas besoin de chercher l'adresse ni de créer l'enveloppe
  - Tu mets simplement la lettre dans sa main
  - De la même façon, Laravel fournit l'objet Request directement à la méthode

Code exemple

🗀 **app/Http/Controllers/UserController.php**

```
<?php
```

```
namespace App\Http\Controllers;
use Illuminate\Http\Request;
class UserController extends Controller
{
   // Méthode pour créer un utilisateur
   // Laravel va automatiquement injecter l'objet Request
   public function store(Request $request)
   {
      // On peut accéder directement aux champs du formulaire
      // Exemple : <input name="name">
      return "Nom reçu : " . $request->name;
   }
}
```

Explication

Ligne	Explication
public function store(Request $request)	Laravel voit Request et crée l'objet automatiquement
$request->name	Permet d'accéder au champ name envoyé par le formulaire
return "Nom reçu : " . $request->name	Pour tester, on renvoie directement la valeur

Test pratique

Crée un formulaire simple :

🗀 **resources/views/users/create.blade.php**

```
<h2>Créer utilisateur</h2>
<form method="POST" action="/users">
   @csrf
   <input type="text" name="name" placeholder="Nom">
   <button type="submit">Envoyer</button>
</form>
```

Envoi du formulaire :

- URL : /users
- Méthode : POST
- Champ name = “Ali”

Résultat attendu :

Nom reçu : Ali

Points importants :

1. Laravel **voit le type** (Request) et **injecte automatiquement l’objet**
2. Pas besoin de créer $request = new Request()
3. On peut accéder **directement aux champs du formulaire**
4. C’est **plus clair, moins de code, facile à tester**

« L’injection par méthode, c’est comme si Laravel te passait directement le courrier prêt à lire, sans que tu aies à préparer l’enveloppe toi-même. »

OFPPT

Formateur : AARAB HAFID